

# How Graphs Can Help You Stay Informed in an Evolving World

MohammadHossein Bateni\*

Google  
New York, New York, USA  
bateni@google.com

Hossein Esfandiari

Google  
London, United Kingdom  
esfandiari@google.com

Lin Chen

Google  
New York, New York, USA  
linche@google.com

Sasan Tavakkol

Google  
Los Angeles, California, USA  
tavakkol@google.com

## Abstract

This study investigates the problem of keeping information up-to-date when crawling data sources that change over time (e.g., websites or location data). Traditional crawling methods often treat data sources independently, making it difficult to capture relationships and propagate updates efficiently. We propose using graph structures to model these relationships and show that, unfortunately, finding the theoretically optimal solution can be intractable. To address this, we introduce a specific graphical model (the latent Bernoulli process model) and demonstrate the complexity of even simple tasks within this framework. We tackle the crawling problem using a reinforcement learning-based algorithm and demonstrate its superiority over traditional baselines on both real and synthetic data. This work highlights the power of graph-structured crawling in helping users stay informed within a dynamic information landscape.

## CCS Concepts

• Information systems → Web crawling.

## Keywords

graph-structured crawling; web crawling; reinforcement learning; information freshness; latent bernoulli process

## ACM Reference Format:

MohammadHossein Bateni, Lin Chen, Hossein Esfandiari, and Sasan Tavakkol. 2026. How Graphs Can Help You Stay Informed in an Evolving World. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3774904.3792942>

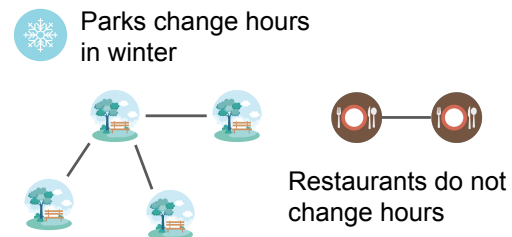
## 1 Introduction

In traditional supervised active learning [6, 12, 18, 19], labels are static. However, in many real-world problems, information changes over time, requiring entities to be "crawled" to keep knowledge up-to-date [2–5, 14, 15, 17]. This "crawling problem" is fundamental to web search engines caching the latest pages and to geographical

\*Authors are listed in alphabetical order.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*  
© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2307-0/2026/04  
<https://doi.org/10.1145/3774904.3792942>



**Figure 1: An example of how parks, which are strongly connected to each other, may change their hours collectively during the winter season, while restaurants, which are not as strongly connected to parks but are strongly connected among themselves, may not change their hours during the winter.**

platforms (e.g., Google Maps, Yelp, etc.) tracking the changing opening hours of locations like parks, which may change seasonally or due to unexpected events.

Previous studies often treat these objects as independent, but this is generally not true. For example, when breaking news occurs, the web pages of most mainstream media outlets report it, causing a group of news websites to update their pages at the same time. Similarly, during a pandemic, parks in a certain area are likely to change their hours together or at the beginning of winter, it is common for parks to change their hours. In Figure 1, we provide an example of how parks, which have strong connections with each other, may change their hours together during the winter season, while restaurants, which are not as strongly connected to parks but are strongly connected to each other, may not change their hours during the winter.

Intuitively, provided that an object updates, its neighbors on the graph have an increased probability of an update. In this work, we propose using a graph structure to model these interactions. Our main contributions are: We formally define the problem of graph-structured crawling and show that learning in the worst case is impossible in polynomial time. We then introduce the latent Bernoulli process model (LBM), show that selecting the best object is #P-complete, and analyze the exponential complexity of its dynamic programming solution. Finally, we propose a scalable reinforcement learning (RL) based algorithm that outperforms strong

baselines on real and synthetic data, demonstrating the value of graph information.

The rest of the paper is organized as follows. Section 2 defines the graph-structured crawling problem and presents a general impossibility result. Section 3 introduces the LBM, analyzes its #P-complete hardness, and details the exponential complexity of a dynamic programming solution. Section 4 proposes our scalable RL-based algorithm. Section 5 presents experimental results, followed by Section 6 (Related Work) and Section 7 (Conclusion). Due to space constraints, proofs of our theoretical results are omitted.

## 2 Graph-Structured Crawling and Its General Hardness

In this section, we introduce the concept of graph-structured crawling. Specifically, we formalize the correlations between the updates of objects in a graph and define the crawling problem on such graphs.

*Updates of graph-structured objects.* We use a counting process to model the updates over time. Recall that a counting process  $\{N(t)\}_{t \geq 0}$  is a stochastic process such that  $N(0) = 0$  and  $N(t)$  is a right-continuous step function with jumps of size +1. Each jump of  $N(t)$  represents an update.

Let  $V$  denote the set of objects of interest, and  $\{N_v(t)\}$  be the counting process of an object  $v \in V$ . We assume that the joint stochastic process  $\{N_V(t)\}_{t \geq 0} \triangleq \{N_v(t)\}_{t \geq 0, v \in V}$  has an (unknown) joint distribution  $Pr(N_v(t) : v \in V)$ , where a graphical model can be introduced. For example, if the objects form a graphical structure  $G = (V, E)$ , the joint probability may be factorized according to the cliques of the graph [13].

*Crawling.* In the crawling problem, a crawler interacts with an environment consisting of a joint stochastic process  $\{N_V(t)\}_{t \geq 0}$ . At each query time  $t_i$  ( $i = 1, 2, 3, \dots$  and  $0 \leq t_1 < t_2 < \dots$ )<sup>1</sup>, the crawler selects an object  $v_i \in V$  and observes whether it has changed since the last time the crawler queried it. The crawler's observation at time  $t_i$  is given by

$$o_i = \mathbb{1} \{N_{v_i}(A(v_i, t_i)) \neq N_{v_i}(t_i)\}. \quad (1)$$

where  $A(v, t)$  is the time of the last query on  $v$  prior to time  $t$  (or 0 if  $v_i$  has never been queried before), and is formally defined by

$$A(v, t) \triangleq \max(\{0\} \cup \{t_j | v_j = v, t_j < t\}). \quad (2)$$

The crawler's reward at time  $t_i$  is determined by the reward function  $r_i = r(v_i, t_i)$ , which could either be equal to  $o_i$  if the crawler only cares about whether it hits an update at time  $t_i$ , or

$$r_i = f\left(\left(t_i - [A(v_i, t_i)]_{N_{v_i}(\cdot)}\right)_+\right) \quad (3)$$

if the crawler also takes into account the freshness of object  $v_i$  at time  $t_i$ , where  $(x)_+ = \max\{x, 0\}$ ,  $f$  is a non-negative non-increasing function,  $[\tau]_{N_v(\cdot)} \triangleq \sup\{t \in (\tau, \infty) | N_v(t) = N_v(\tau)\}$  is the next jumping time of  $N_v(\cdot)$  at time  $\tau$ .

The aim of the crawler is to maximize either the expected average return over  $T$  rounds,  $\mathbb{E}[AR_T]$ , if the game lasts for a fixed number of rounds, or the expected limiting average return,  $\mathbb{E}[AR_\infty]$ , if the

<sup>1</sup>The crawler can choose the query time  $t_i$  adaptively, potentially subject to a constraint such as a minimum time between queries ( $t_{i+1} - t_i \geq 1$ ), or the query times can be pre-determined.

game is ongoing indefinitely. The  $T$ -round average return is defined as  $AR_T = \frac{1}{T} \sum_{i=1}^T r_i$ , and the limiting average return is defined as  $AR_\infty = \liminf_{T \rightarrow \infty} AR_T$ .

### 2.1 General Impossibility Result

We first demonstrate that in the worst case, it is not possible to learn anything meaningful in polynomial time, despite the presence of a learnable structure that requires exponential time to uncover. Mathematically, we prove that there exists a process model such that  $\mathbb{E}[AR_T] \in o(1)$  for any polynomial  $T$ , while  $\mathbb{E}[AR_\infty] = 1$ . This result motivates us to investigate specific stochastic process models in the subsequent sections.

**Theorem 1.** *Let us set the reward function*

$$r_i = f\left(\left(t_i - [A(v_i, t_i)]_{N_{v_i}(\cdot)}\right)_+\right) \quad (4)$$

as in equation 3, where  $f(t) = e^{-t}$ . For any algorithm, there exists a process (unknown to the algorithm) over  $n$  objects such that  $\mathbb{E}[AR_T] = o(1)$  for any polynomial  $T$ , while  $\mathbb{E}[AR_\infty] = 1$ .

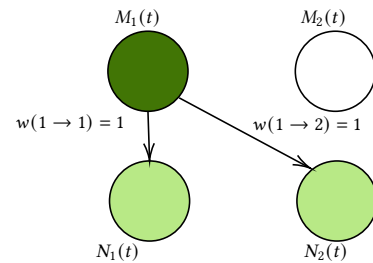
## 3 Latent Bernoulli Process Model and Its Complexity

In light of the general intractability, we introduce the Latent Bernoulli Process Model (LBM) to capture properties observed in real-world crawling, such as simultaneous updates. We assume time is discrete. At each time step  $t \in \mathbb{N}$ , each object  $v$  has an unobservable latent Bernoulli process  $M_v(t)$  which updates with probability  $\lambda_v(t)$ . This latent update can then propagate to observable objects  $N_v(t)$  based on a weighted graph structure  $G = (V, E)$ .

The weight  $w(u \rightarrow v) \in [0, 1]$  is the probability of the latent process  $M_u(t)$  influencing (transferring an arrival to)  $N_v(t)$ . Formally, the observable update  $\Delta N_v(t) = N_v(t) - N_v(t-1)$  is given by

$$\Delta N_v(t) = \bigvee_u B_{u \rightarrow v}(t) \Delta M_u(t), \quad (5)$$

where  $\Delta M_u(t) \sim \text{Bern}(\lambda_u(t))$  and  $B_{u \rightarrow v}(t) \sim \text{Bern}(w(u \rightarrow v))$ . This is not a diffusion process; as shown in Figure 2, related objects receive an update simultaneously.



**Figure 2: Example of a latent Bernoulli process.**

*Complexity Analysis.* Despite its specific structure, the LBM remains computationally challenging. We demonstrate that selecting the optimal object, even in the final step, is #P-complete.

The SELECTBESTOBJECT (SBO) Problem is given the history  $H_{t-1}$ , model parameters  $(\lambda, w)$ , and involves finding the object  $v$  that

maximizes the expected reward  $\mathbb{E}[r(v, T+1) \mid H_{t-1}]$  at the final step.

**Theorem 2.** *The SBO problem is #P-complete, even when  $w(u \rightarrow v) \in \{0, 1\}$  for every pair of objects  $u, v$  and  $r(v, t) = 1$  for all  $t \in [T]$ .*

A standard dynamic programming approach (Algorithm 1) to find the optimal policy for a finite horizon  $T$  is intractable. The randomness is determined by  $O(|V|^2 T)$  Bernoulli random variables. Computing the probability  $p_j(v)$  (Line 2) requires summing over all combinations of these variables, leading to an intractable exponential time complexity of  $O(2^{|V|^2 T^2 |V|})$ . This motivates our scalable, approximate solution.

---

**Algorithm 1** DYNAMIC PROGRAMMING at the  $t$ -th step (DP <sub>$t$</sub> )

---

**Input:**  $H_{t-1} = \{(v_i, r_i)\}_{1 \leq i \leq t-1}$

```

1: for all  $v \in V, j \in \{0, 1\}$  do
2:    $p_1(v) \leftarrow \Pr(r_t = 1 \mid H_{t-1}, v_t = v); p_0(v) \leftarrow 1 - p_1(v)$ 
3:   if  $t < T$  then  $q_j(v) \leftarrow \text{DP}_{t+1}(H_{t-1}, v_t = v, r_t = j)$ 
4:   else  $q_j(v) \leftarrow 0$ 
5:   end if
6: end for
7:  $v \leftarrow \arg \max_{v \in V} p_1(v)(1 + q_1(v)) + p_0(v)(0 + q_0(v))$ 
8: return  $v$  and the maximum value  $Q_t$ 
```

---

#### 4 Reinforcement Learning-Based Crawler

Given the intractability of the optimal DP solution, we propose a reinforcement learning-based (RL-based) algorithm. We define the state space as  $\mathcal{S} = \mathcal{S}_{\text{node}}^V$ , the Cartesian product of the state spaces of all objects. We divide the state space  $\mathcal{S}_{\text{node}}$  of a single object into two components  $\mathcal{S}_{\text{node}} = \mathcal{S}_{\text{history}} \times \mathcal{S}_{\text{graph}}$  where  $\mathcal{S}_{\text{history}}$  denotes the history-encoding state and  $\mathcal{S}_{\text{graph}}$  denotes the graph-encoding state. We present two versions of the RL-based algorithm: one using both history and graph states, and one (for comparison) using only history. We want to show that the graphical structure can improve performance by exploiting the intuition that neighboring objects update concurrently.

**Designing graph-encoding state.** The graph state  $\mathcal{S}_{\text{graph}}$  can be encoded in several ways, such as using graph embeddings, the object's row in the adjacency matrix, or a vector of distances to all other objects.

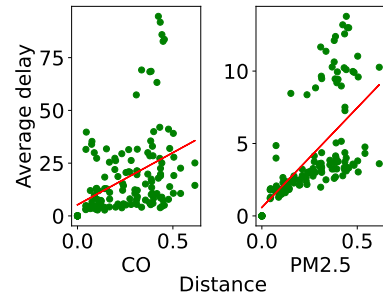
**Designing history-encoding state.** We design the history-encoding state  $\mathcal{S}_{\text{history}} = \{0, 1\} \times \mathbb{N} \times \mathbb{N} \times \mathbb{R} \times \mathbb{R}$ . This 5-dimensional vector tracks: (1) whether the object has ever been queried; (2) the time elapsed since the last query that resulted in an update; (3) the time elapsed since the last query that resulted in no update; and (4, 5) two features representing recent query outcomes, which are set to a high value (e.g., 100) on a query and decay over time. In our proposed RL-based approach, we utilize the double Deep Q-Network (double DQN) algorithm [20]. The discount value is set to  $\gamma = 0.7$ , and we use a two-layer neural network with widths of 75 and 40 as our Q network and the target Q network.

#### 5 Numerical Studies

**Datasets.** We evaluate our RL-based algorithm for graph-structured crawling on the Beijing air quality dataset [22], visiting monitoring

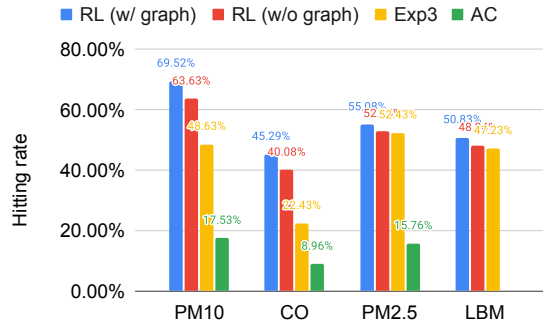
sites to track PM10, CO, and PM2.5 values. We also examine the proposed RL-based method on the latent Bernoulli model using a graph structure generated by an Erdős-Rényi model.

**Verification of Intuition behind Graph-Structured Crawling.** We aim to verify the intuition that updates in an object's neighbors are likely to occur following an update in the object. To test this, we use the Beijing Multi-Site Air-Quality Dataset. For each pair of monitoring sites  $u$  and  $v$ , we measure the delay in transferring an update from  $u$  to  $v$  by calculating the time difference between an update at  $u$  and the next update at  $v$ . We then compute the average delay for each pair of sites and plot a scatter plot in Figure 3, with the horizontal coordinate representing the distance between the sites and the vertical coordinate representing the average delay. A regression line is also plotted. The results show a positive correlation between distance and average delay, indicating that updates in closer sites are more likely to occur simultaneously.



**Figure 3:** A scatter plot illustrating the correlation between the average delay in transferring updates and the distance between monitoring sites in the Beijing Multi-Site Air-Quality Dataset. The left and right panels depict the results for updates of CO and PM2.5 respectively, when the threshold is exceeded. A regression line is also shown, which demonstrates a positive correlation between the distance and average delay, thus supporting the intuition behind graph-structured crawling.

**Performance Evaluation of Crawling Algorithms.** We compare the RL-based algorithm to two baseline algorithms: Exp3 [1, 16], an adversarial bandit algorithm, and a variant of the active covering algorithm (AC) [12]. Since the original AC assumes static labels, our variant restarts the algorithm periodically once all objects have been queried. For the Beijing Multi-Site Air-Quality Dataset, the graph-encoding state is the site's distance from all other sites. We use the first 34000 records for training and evaluate on the following 1000 records. For the latent Bernoulli model, we use the corresponding row in the adjacency matrix. To evaluate performance, we calculated their hitting rates (the percentage of queries that find an update). We present the results in Figure 4 and Table 1. The performance of our RL-based algorithm is dominant, consistently outperforming all baselines across all tasks. The advantage is particularly stark in the real-world PM10 and CO tasks, where our full model (RL w/ graph) achieves 69.5% and 45.3% hitting rates, respectively, eclipsing the strong Exp3 baseline (48.6% and



**Figure 4: The chart illustrates the performance comparison of the RL-based algorithm with the baselines in terms of hitting rate. The LBM column represents the results for the latent Bernoulli model.**

22.4%). To isolate the value of the graph structure, we also evaluated an ablation (RL w/o graph). This model, which uses our novel state representation without graph data, still significantly outperforms Exp3 (63.6% vs 48.6% on PM10). The further improvement to 69.5% when adding the graph confirms that both our state design and the graph structure contribute substantial value. All RL and Exp3 methods strongly outperformed the AC variant. Our method also achieved the highest performance on the synthetic LBM data, confirming its robustness.

| Task  | RL (w/ graph) | RL (w/o graph) | Exp3   | AC     |
|-------|---------------|----------------|--------|--------|
| PM10  | <b>69.52%</b> | 63.63%         | 48.63% | 17.53% |
| CO    | <b>45.29%</b> | 40.08%         | 22.43% | 8.96%  |
| PM2.5 | <b>55.08%</b> | 52.85%         | 52.43% | 15.76% |
| LBM   | <b>50.83%</b> | 48.24%         | 47.23% | –      |

**Table 1: Hitting rate of the RL-based algorithms and the baselines. The LBM rows represent the results for the latent Bernoulli model.**

## 6 Related Work

Cho and Garcia-Molina [7, 8] proposed refresh policies, studying their effectiveness with two freshness metrics and a Poisson process change model. Cho and Garcia-Molina [9] focused on estimating the Poisson rate within this model. Azar et al. [2] formalized the optimization problem of finding a stationary policy under request rate and utility models, aiming for optimal freshness. Han et al. [11] used five base crawling algorithms (including [2]) and employed multi-armed bandit methods to select the best algorithm at each crawling step. Kolobov et al. [15] extended the results of Azar et al. [2] by considering politeness constraints on request rates. For a web crawling survey, see Olston et al. [17]. While seemingly similar, [21] focused on mirroring online social networks for data collection, not tracking online content updates. [10] investigated agent movement within a growing web graph, a distinct problem.

## 7 Conclusion

We studied crawling in dynamic environments. We proposed a graph structure to model object relationships and capture influence. Learning the structure was intractable in polynomial time. We introduced the latent Bernoulli process model, but even selecting the best object was #P-complete. We analyzed a dynamic programming solution, highlighting its exponential complexity. This theoretical analysis motivated our proposed reinforcement learning-based algorithm, which we evaluated on real and synthetic data, showing it outperforms strong baselines. Real data analysis confirmed our intuition about graph-structured correlations.

## References

- [1] Peter Auer, Nicolo Cesa-Bianchi, Yoav Freund, and Robert E Schapire. 1995. Gambling in a rigged casino: The adversarial multi-armed bandit problem. In *Proceedings of IEEE 36th annual foundations of computer science*. IEEE, 322–331.
- [2] Yossi Azar, Eric Horvitz, Eyal Lubetzky, Yuval Peres, and Dafna Shahaf. 2018. Tractable near-optimal policies for crawling. *Proceedings of the National Academy of Sciences* 115, 32 (2018), 8099–8103.
- [3] Melih Bastopcu and Sennur Ulukus. 2020. Information freshness in cache updating systems. *IEEE Transactions on Wireless Communications* 20, 3 (2020), 1861–1874.
- [4] Melih Bastopcu and Sennur Ulukus. 2020. Who should Google scholar update more often?. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 696–701.
- [5] Melih Bastopcu and Sennur Ulukus. 2021. Age of information for updates with distortion: Constant and age-dependent distortion constraints. *IEEE/ACM Transactions on Networking* 29, 6 (2021), 2425–2438.
- [6] Lin Chen, Hamed Hassani, and Amin Karbasi. 2017. Near-optimal active learning of halfspaces via query synthesis in the noisy setting. In *Thirty-First AAAI Conference on Artificial Intelligence*.
- [7] Junghoo Cho and Hector Garcia-Molina. 2000. Synchronizing a database to improve freshness. *ACM sigmod record* 29, 2 (2000), 117–128.
- [8] Junghoo Cho and Hector Garcia-Molina. 2003. Effective page refresh policies for web crawlers. *ACM Transactions on Database Systems (TODS)* 28, 4 (2003), 390–426.
- [9] Junghoo Cho and Hector Garcia-Molina. 2003. Estimating frequency of change. *ACM Transactions on Internet Technology (TOIT)* 3, 3 (2003), 256–290.
- [10] Colin Cooper and Alan Frieze. 2002. Crawling on web graphs. In *Proceedings of the thirty-fourth annual ACM symposium on Theory of computing*. 419–427.
- [11] Shuguang Han, Michael Bendersky, Przemek Gajda, Sergey Novikov, Marc Najork, Bernhard Brodowsky, and Alexandrin Popescul. 2020. Adversarial Bandits Policy for Crawling Commercial Web Content. In *Proceedings of The Web Conference 2020*. 407–417.
- [12] Heinrich Jiang and Afshin Rostamizadeh. 2021. Active Covering. In *International Conference on Machine Learning*. PMLR, 5013–5022.
- [13] Daphne Koller and Nir Friedman. 2009. *Probabilistic graphical models: principles and techniques*. MIT press.
- [14] Andrey Kolobov, Sébastien Bubeck, and Julian Zimmert. 2020. Online learning for active cache synchronization. In *International Conference on Machine Learning*. PMLR, 5371–5380.
- [15] Andrey Kolobov, Yuval Peres, Eyal Lubetzky, and Eric Horvitz. 2019. Optimal freshness crawl under politeness constraints. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 495–504.
- [16] Tor Lattimore and Csaba Szepesvári. 2020. *Bandit algorithms*. Cambridge University Press.
- [17] Christopher Olston, Marc Najork, et al. 2010. Web crawling. *Foundations and Trends® in Information Retrieval* 4, 3 (2010), 175–246.
- [18] Burr Settles. 2012. Active learning. *Synthesis lectures on artificial intelligence and machine learning* 6, 1 (2012), 1–114.
- [19] Simon Tong. 2001. *Active learning: theory and applications*. Stanford University.
- [20] Hado Van Hasselt, Arthur Guez, and David Silver. 2016. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 30.
- [21] Shaozhi Ye, Juan Lang, and Felix Wu. 2010. Crawling online social graphs. In *2010 12th International Asia-Pacific Web Conference*. IEEE, 236–242.
- [22] Shuyi Zhang, Bin Guo, Anlan Dong, Jing He, Ziping Xu, and Song Xi Chen. 2017. Cautionary tales on air-quality improvement in Beijing. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 473, 2205 (2017), 20170457.